

Exactly. 🔥 This is the right structure, and we'll stick to it.

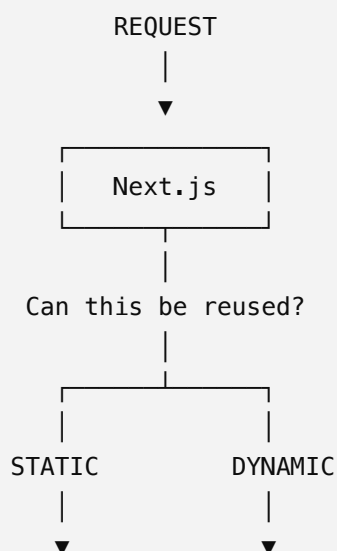
We've already built a lot of the **cache pieces**, so the next step is to connect them into the actual decision-making process.

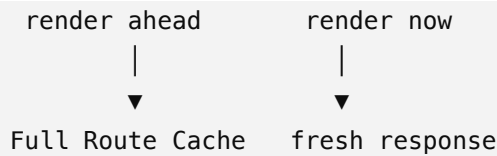
## 🧠 Layer 1 — Static vs Dynamic Rendering

The first question Next.js has to answer is:

**"Can I generate this route ahead of time and reuse the result, or do I need to render it for each request?"**

Think of the pipeline as:





But "static vs dynamic" is NOT simply determined by whether you use `fetch()`.

There are several signals.

## 1. The important signals

We'll go through these one by one:

### A. `fetch()`

```
const products = await fetch('/api/products')
```

The caching behavior of the fetch matters.

For example:

```
fetch(url, {
  cache: 'no-store'
})
```

means:

Don't reuse this fetched result from the Data Cache.

Whereas a cacheable fetch can allow the result to be reused.

### B. `cookies()`

```
const cookieStore = await cookies()
```

This accesses **request-specific information**.

For example:

```
Request 1 → user=A  
Request 2 → user=B
```



The result can differ between requests.

So traditionally, this is a signal that the route needs request-time rendering.

### C. `headers()`

Same idea:

```
const headersList = await headers()
```

Headers belong to the incoming request.

For example:

```
Authorization  
User-Agent  
Accept-Language
```



Those can differ from request to request.

### D. `searchParams`

For example:

```
/products?category=shoes
```



Your page can receive:

```
export default async function Page({ searchParams }) {  
  // ...  
}
```

The URL itself contains request-specific information.

So we need to understand exactly **when** `searchParams` **affects rendering and caching**.

## E. `revalidate`

Example:

```
export const revalidate = 3600
```

This means:

The generated route/data can be reused, but it should be considered stale after a certain period.

This leads us directly into **ISR**.

```
Generate
  ↓
Cache
  ↓
Reuse
  ↓
becomes stale
  ↓
regenerate
```



## F. `generateStaticParams()`

This is especially important for dynamic routes.

Imagine:

```
/products/[id]
```



You could tell Next.js ahead of time:

```
export async function generateStaticParams() {
  return [
    { id: '1' },
    { id: '2' },
    { id: '3' }
  ]
}
```

Now Next.js knows some concrete routes that can be generated ahead of time.

```
/products/1  
/products/2  
/products/3
```



We'll go deeply into **what happens for** `/products/999` too.

## 🔥 But here's the important distinction

Don't mix these two questions:

### Question 1

Is the **DATA** cached?

That's primarily about the:

Data Cache



### Question 2

Is the rendered **ROUTE** cached?

That's about:

Full Route Cache



These are related, but **not the same thing**.

For example:

```
Database  
  ↓  
fetch()  
  ↓  
Data Cache  
  ↓  
React/Next rendering  
  ↓  
HTML + RSC Payload  
  ↓  
Full Route Cache
```



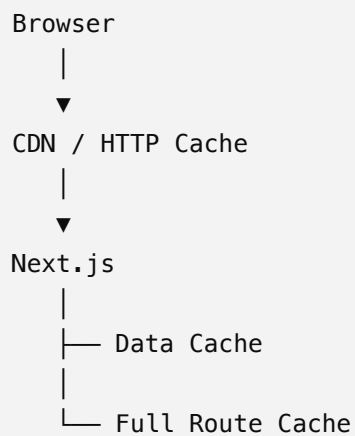
That's one of the most important relationships we'll keep returning to.



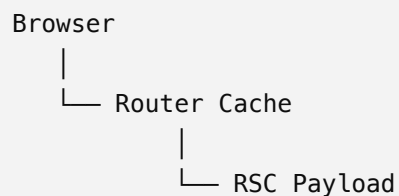
## And then there's the browser

After Next.js has produced the response, we aren't done.

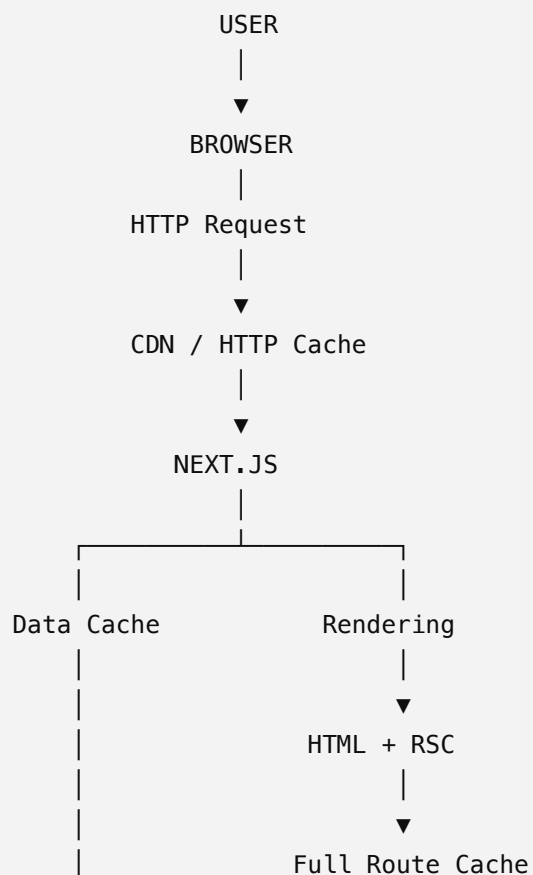
We potentially have:

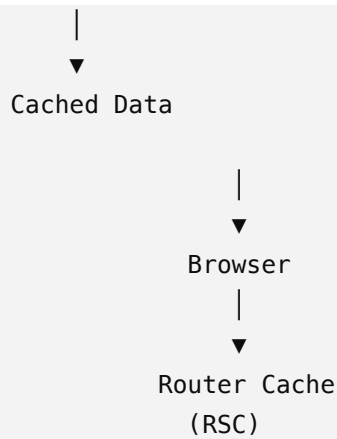


And after navigation:



So eventually our complete mental model becomes:





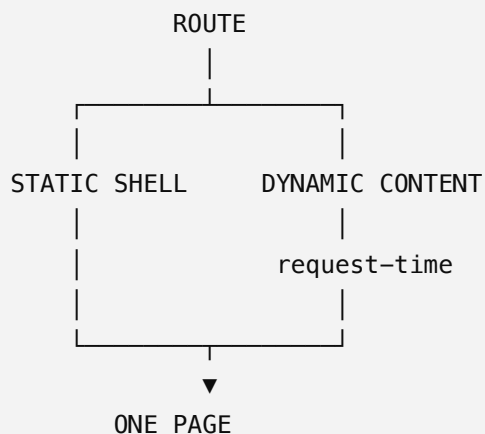
And **Layer 2** will later modify our understanding of the middle part:

Traditional:

Static route  
OR  
Dynamic route



versus the newer idea:



That's where **Cache Components** + `use cache` + **Suspense** + **PPR** become really interesting.

## So let's start Layer 1 properly

Our next lesson should be:

### Lesson 1 — How Next.js determines Static vs Dynamic

We'll take **the exact same page** and progressively change it:

```
// Version 1
export default async function Page() {
  return <h1>Hello</h1>
}
```



```
// Version 2
await fetch(...)
```



```
// Version 3
fetch(..., { cache: 'no-store' })
```



```
// Version 4
cookies()
```



```
// Version 5
headers()
```



```
// Version 6
searchParams
```



```
// Version 7
revalidate
```

And for **each version**, we'll answer the same 6 questions:

| Question                       | We'll determine                    |
|--------------------------------|------------------------------------|
| When does rendering happen?    | Build / request / revalidation     |
| Is data cached?                | Yes / No                           |
| Is the route cached?           | Yes / No                           |
| What happens to HTML?          | Reused / regenerated               |
| What does the browser receive? | HTML / RSC / both                  |
| Can CDN reuse it?              | Under what HTTP caching conditions |

That repetition is deliberate—it'll make the architecture **click instead of becoming a pile of Next.js rules.** 🧠🔥

Let's start with **Version 1: the completely static page**, and trace it from **Browser** → **CDN** → **Next.js** → **rendering** → **Full Route Cache** → **Browser** .



Today 4:48 AM

ok

## Layer 1 — Lesson 1: The Completely Static Page

Let's start with the **simplest possible Next.js page** and trace every step.

 TypeScript



```
export default function Page() {  
  return <h1>Hello</h1>  
}
```

Assume this is:

app/page.tsx



So the URL is:

/



## 1 First question: Is this static or dynamic?

This page:

`</>` TypeScript



```
export default function Page() {  
  return <h1>Hello</h1>  
}
```

doesn't depend on:

- cookies
- headers
- request-specific data
- search params
- database data
- uncached request-time APIs

It always produces:

`</>` HTML



```
<h1>Hello</h1>
```

Therefore, in the traditional Layer-1 mental model, this is a static route.

The key idea:

If the output can be determined ahead of time and reused for multiple requests, Next.js can pre-render it.

## 2 When does rendering happen?

This is the important part.

You run:

`</>` Bash



```
next build
```

Next.js analyzes the route and generates its static output.

Conceptually:



```
next build
  ↓
Next.js renders /
  ↓
React Server Components execute
  ↓
HTML is produced
  +
RSC Payload is produced
  ↓
Full Route Cache
```

So the page **doesn't need to execute the entire rendering process every time someone requests** / .

That's the whole benefit of static rendering.

### 3 What exactly gets generated?

Remember our earlier discussion:

**Full Route Cache stores the rendered result.**

Conceptually:



Full Route Cache

```
HTML
<h1>Hello</h1>
RSC Payload
```

So don't think:

"Full Route Cache stores the React component."

It doesn't.

It stores the **result of rendering the route**.

Think:



Component

↓

```
Rendering
  ↓
Rendered output
  ↓
Full Route Cache
```

## 4 Now a user visits /

The browser sends:

```
GET /
```



Conceptually:

```
Browser
  |
  | GET /
  ▼
Next.js infrastructure
```



But here's where your previous CDN question becomes important.

## Does the request always go directly to Next.js?

No.

If your deployment has a CDN/HTTP cache in front of Next.js, the request may encounter the CDN first.

So:

```
Browser
  |
  ▼
CDN
  |
  ├── HIT → return cached HTTP response
  |
  └── MISS → forward request
        |
        ▼
      Next.js
```



This is a **different cache layer** from Next.js's Full Route Cache.

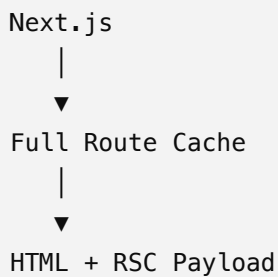
## 5 Let's assume CDN MISS

Suppose this is the first request.



Next.js needs to obtain the route response.

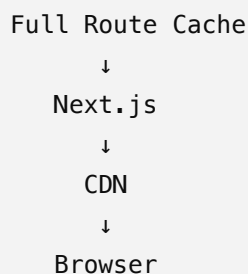
Because `/` is statically rendered, Next.js can use the previously generated route result.



So Next.js doesn't have to render `<Page />` from scratch for this request.

## 6 Next.js sends the response

The response goes back:



The browser initially gets the document response containing the HTML.

Conceptually:

```
<html>
...
```

```
<body>
  <h1>Hello</h1>
</body>
</html>
```

The browser can therefore **display "Hello" immediately from the HTML.**

This is one of the big advantages of SSR/SSG-style rendering compared with a traditional client-only React app.

## 7 Where does the RSC Payload fit?

Now let's slow down here because this is where people often get confused. 🤔

Next.js isn't only thinking in terms of:

HTML



For the App Router, there's also:

RSC Payload



The RSC Payload represents the rendered **React Server Component tree/data needed by the client.**

For the initial navigation, the browser can receive the information needed to hydrate/continue the App Router experience.

So conceptually:

Full Route Cache



But don't interpret that as:

"The browser always receives the Full Route Cache itself."

No.

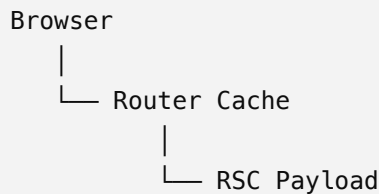
The cache is **server-side infrastructure.**

The browser receives an **HTTP response** produced from that cached result.

## 8 Then what is Router Cache?

This is a completely different cache.

After the App Router receives the RSC information, the client can keep relevant RSC payloads in memory:



Why?

Imagine the user navigates:

```
graph TD; A[" / "] --> B[" /products "]; B --> C[" /products/123 "]; C --> D[" / "];
```

A vertical sequence of navigation steps connected by downward arrows:   
1. /   
2. /products   
3. /products/123   
4. /

The client may already have the RSC payload for / .

Instead of necessarily going all the way through:

```
graph TD; Browser1[Browser] --> CDN[CDN]; CDN --> Nextjs[Next.js]; Nextjs --> FullRouteCache[Full Route Cache]; FullRouteCache --> Browser2[Browser];
```

A vertical flow diagram showing the request path:   
1. Browser   
2. ↓   
3. CDN   
4. ↓   
5. Next.js   
6. ↓   
7. Full Route Cache   
8. ↓   
9. Browser

the Router Cache can allow the client-side router to reuse information it already has.

So:

### Full Route Cache

SERVER



Rendered route

HTML + RSC Payload



## Router Cache

CLIENT / BROWSER



RSC Payload



That's the distinction we were discussing earlier.

## 9 Now let's compare all the layers

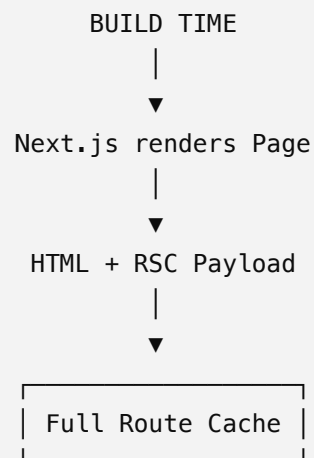
For our tiny page:

```
</> TypeScript
```

```
export default function Page() {  
  return <h1>Hello</h1>  
}
```



we can draw the entire journey:



REQUEST TIME

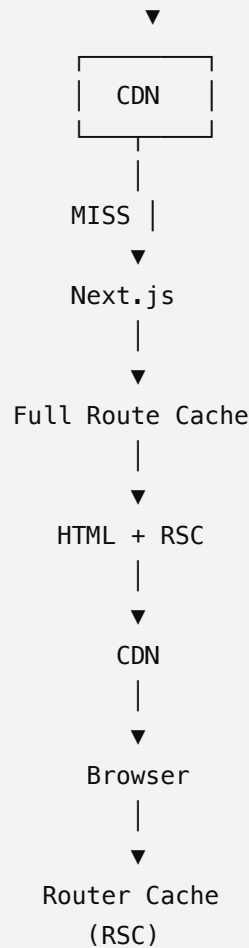


Browser



GET /





🔥 That's our first complete Layer-1 flow.

## 10 One VERY important distinction

You asked earlier:

"Why do we need CDN cache when we already have Next.js cache?"

This example gives us the answer.

They're caching **different things at different locations**.

### Full Route Cache

Where?

→ Next.js/server infrastructure

What?

→ Rendered route result

→ HTML + RSC Payload

### CDN / HTTP Cache

Where?  
→ Edge/CDN infrastructure



What?  
→ HTTP response

## Router Cache

Where?  
→ User's browser



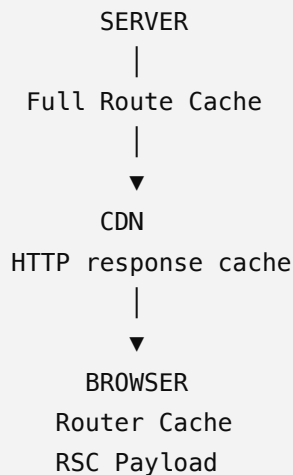
What?  
→ RSC Payload

So it's not:

Three caches doing exactly the same job



It's more like:



Each sits at a **different point in the request path**.

## The 6 answers for Version 1

### Question

### Static page

When does rendering happen?

Ahead of requests, during build

Is data cached?

No separate data fetch here

Is route cached?

✅ Full Route Cache

| Question                   | Static page                                                                         |
|----------------------------|-------------------------------------------------------------------------------------|
| What happens to HTML?      | Generated ahead of time and reusable                                                |
| What does browser receive? | HTTP response containing the rendered HTML; App Router also uses RSC information    |
| Can CDN reuse it?          | <b>Potentially yes</b> , if the deployment/response's HTTP caching policy allows it |

And the core mental model is:

**Static rendering means Next.js can produce the route result ahead of the user's request. The Full Route Cache can then reuse that rendered result. A CDN may independently cache the HTTP response, and the browser has its own Router Cache for RSC payloads.**

## Next: Version 2

Now we make **one tiny change**:

 TypeScript



```
export default async function Page() {  
  const res = await fetch("https://api.example.com/products")  
  
  const products = await res.json()  
  
  return <ProductList products={products} />  
}
```

And this is where things get interesting because we'll discover:

**"I used `fetch()` " does NOT automatically mean "dynamic."**

We'll trace **cacheable `fetch()` → Data Cache → Full Route Cache → CDN → Router Cache** next.





















































































































































































































































































































































Ahhh, yes — I see the exact confusion now. 😊 You're asking:

"If `/products` is cached and doesn't depend on the cookie, and I change a cookie in the browser, how does the server ever learn that the cookie changed?"

The answer is: **it learns it on the next request that actually reaches the server.** And if that request is satisfied entirely by a CDN cache, the origin server may **not need to know at all.**

Let's separate these cases.

---

## 1. Cookie changes in the browser

Suppose:

Browser

`theme = light`



Server Action changes it:

```
cookies().set("theme", "dark")
```



Response:

```
</> http
```



```
Set-Cookie: theme=dark
```

Browser now has:

```
Browser
```



```
theme = dark
```

At this point:

```
Server: ❌ hasn't received "theme=dark" as a new request
```



```
Browser: ✅ knows theme=dark
```

---

## 2. Then user requests `/products`

Browser makes:

```
</> http
```



```
GET /products
```

```
Cookie: theme=dark
```

Now there are two possibilities.

### Case A — CDN handles it

```
Browser
```



```
| GET /products
```

```
| Cookie: theme=dark
```



```
CDN
```

```
| Cache HIT
```



```
Cached /products response
```

The request **doesn't reach Next.js origin**.

Therefore:

Next.js server

✗ doesn't learn about this request

✗ doesn't need to



The CDN has already answered the request.

And that's perfectly fine if `/products` doesn't care about `theme`.

### 3. So when does Next.js actually learn the new cookie?

When there is a request that **reaches Next.js**.

For example:

Browser



Cookie: theme=dark



CDN



Cache MISS



Next.js



cookies()



Now Next.js can see:

theme=dark



So:

Cookie changed



Browser stores new cookie



Future HTTP request



Cookie automatically attached



If request reaches server



↓  
Server can read new cookie

## 4. But here's the important realization 💡

You don't actually need to **"tell the server that the cookie changed"** as a separate operation.

The cookie itself is the state.

The browser effectively says:

"From now on, whenever I make an applicable request, I'll send `theme=dark`."

So imagine:

10:00

Browser:  
`theme=light`



Server Action:

Set-Cookie: `theme=dark`



Browser:

`theme=dark`



Then:

10:05  
GET /profile  
Cookie: `theme=dark`



Server sees it.

There was no separate:

UPDATE\_COOKIE /api/notify-server



required.

## 5. What if `/products` is completely cached?

Suppose:

`/products`



doesn't care about cookies.

CDN has:

`/products → Product HTML`



User requests it:

```
Browser
Cookie: theme=dark
  |
  ▼
CDN
  |
  ▼
Cached Product HTML
```



Next.js doesn't learn about the cookie.

**And that's okay.**

Because the page doesn't use it.

It's exactly like sending someone a letter that says:

"Here is my favorite color: dark"



but the person only needs to answer:

"What products are available?"



They don't need to process your favorite color. 😊

---

## 6. Now compare with `/profile`

Suppose `/profile` says:

`</> TypeScript`



```
const cookieStore = await cookies()
const theme = cookieStore.get("theme")?.value
```

Now the cookie matters.

Browser:

```
</> http
GET /profile
Cookie: theme=dark
```

If CDN cannot safely serve a generic cached response:

```
Browser
  ↓
CDN
  ↓
Next.js
  ↓
cookies()
  ↓
theme = dark
  ↓
render
```

Now Next.js **does** learn about it.

---

## 7. There's one more scenario: you WANT the server to know immediately

Suppose you have some client-side cookie that you change using JavaScript:

```
</> JavaScript
document.cookie = "theme=dark"
```

The server isn't immediately notified.

If you specifically need the server to know immediately, you make a request:

```
</> JavaScript
document.cookie = "theme=dark"

await fetch("/api/theme", {
```

```
method: "POST"  
})
```

Now:

```
Browser  
|  
| Cookie already changed  
| Cookie: theme=dark  
▼  
POST /api/theme  
▼  
Next.js  
|  
▼  
Server knows theme=dark
```

But normally you don't need this extra request unless your application has a reason to synchronize server-side state.

## This is the BIG distinction

There are actually **three events**:

### Event 1 — Cookie changes

```
Server → Set-Cookie → Browser
```

or JavaScript changes it.

### Event 2 — Browser makes a request

```
Browser → HTTP Request + Cookie → network
```

### Event 3 — Somebody processes that request

Could be:

```
Browser  
↓  
CDN  
↓  
response
```

or

Browser  
↓  
CDN  
↓  
Next.js



If CDN answers it:

Next.js doesn't see it.



If CDN forwards it:

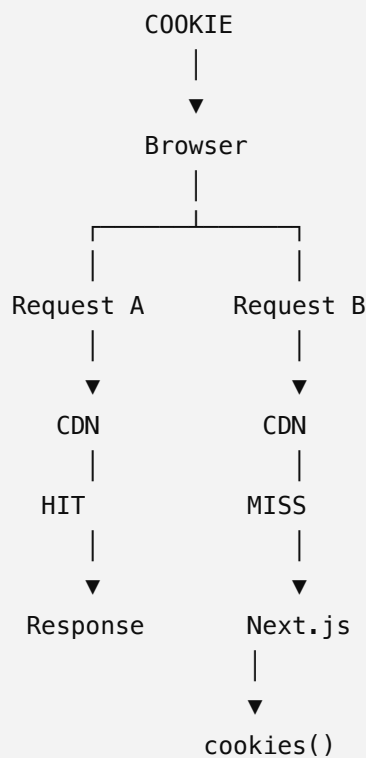
Next.js sees the cookie.



## The sentence that should make this click

A cookie is not a live connection between the browser and server. It's client-side state that gets included in applicable HTTP requests.

So:



In **Request A**, Next.js never knows the cookie existed.

In **Request B**, Next.js sees the cookie.

And that's completely normal. 👍

The next thing that will make this whole topic click is understanding **what exactly a CDN cache key is and whether cookies are part of that key**—because that determines whether two users with different cookies can receive the same cached response.



ok

Perfect 🤔 Let's lock this piece in.

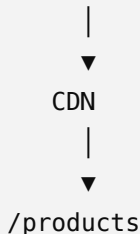
## 🧠 Next: CDN Cache Key + Cookies

The question we'll answer is:

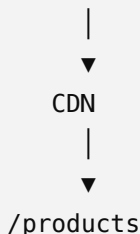
**If two users request `/products` but have different cookies, how does the CDN know whether it can give them the same cached response?**

We'll trace:

User A  
Cookie: theme=dark



User B  
Cookie: theme=light



Then we'll understand:

1. What a cache key actually is
2. Why `/products` normally maps to one cached response
3. How `Vary` works
4. Whether cookies can be part of cache variation
5. Why personalized pages generally shouldn't be blindly CDN-cached

6. How this differs from **Next.js Data Cache / Full Route Cache / Router Cache**

7. Finally, we'll return to our **Layer 1 static vs dynamic exercise**. 🚀



ok

Alright 😊 let's do it. This is one of those concepts that makes **CDN caching + cookies** suddenly click.

## 🌐 **CDN Cache Key — What actually identifies a cached response?**

Imagine the CDN receives:

```
</> http
GET /products
```



The CDN needs to answer:

**"Do I already have a response for this request?"**

It uses something called a **cache key**.

A simplified cache key might be:

```
https://example.com/products
```



So:

```
Request
GET /products
  ↓
Cache Key
example.com/products
  ↓
CDN Cache
  ↓
Cached Response
```



# 1. Two users request the same URL

Suppose:

## User A

```
GET /products
Cookie: theme=dark
```



## User B

```
GET /products
Cookie: theme=light
```

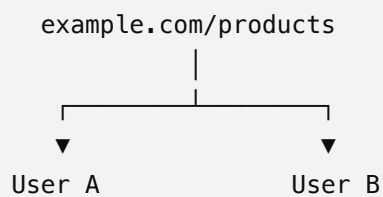


If the CDN's cache key is simply:

```
example.com/products
```



then both requests map to:



So the CDN could potentially return the **same cached response** to both.

That's perfectly fine **if the response doesn't depend on the cookie**.

## 2. Example where this is perfectly safe

Your `/products` page:

</> TypeScript



```
export default async function ProductsPage() {
  const products = await getProducts()

  return (
    <ul>
      {products.map(product => (
        <li key={product.id}>{product.name}</li>
      ))}
    </ul>
  )
}
```

```
)  
}
```

The page doesn't care about:

```
theme  
session  
language  
user ID
```



So:

```
User A  
Cookie: theme=dark  
  |  
  ▼  
/products  
  |  
  ▼  
CDN  
  |  
  ▼  
Cached products response
```



User B:

```
User B  
Cookie: theme=light  
  |  
  ▼  
/products  
  |  
  ▼  
CDN  
  |  
  ▼  
SAME cached products response
```



That's fine.

Because:

```
Cookie —X→ Products HTML
```



The cookie doesn't influence the response.

### 3. Now let's make the cookie matter

Suppose:

`</>` TypeScript

```
export default async function ProductsPage() {  
  const cookieStore = await cookies()  
  
  const theme = cookieStore.get("theme")?.value  
  
  return (  
    <div className={theme}>  
      Products  
    </div>  
  )  
}
```

Now:

Cookie  
↓  
theme  
↓  
HTML

User A:

theme=dark  
↓  
<div class="dark">

User B:

theme=light  
↓  
<div class="light">

So one cached response cannot represent both.

---

### 4. Here's the problem with a naive CDN

Suppose CDN already has:

Cache:

```
/products  
↓  
<div class="dark">
```

User B arrives:

```
</> http  
  
GET /products  
Cookie: theme=light
```



If CDN says:

"Oh, /products is cached! I'll return it."

then:

```
User B  
theme=light  
  
↓  
  
CDN  
  
↓  
  
<div class="dark">
```



🔥 Wrong response.

That's a **cache correctness problem**.

---

## 5. So how does a CDN handle this?

There are several mechanisms in real systems.

One important HTTP mechanism is:

```
</> http  
  
Vary
```



For example:

```
</> http
```



Vary: Accept-Language

This tells caches:

"The response varies based on this request header."

So conceptually:

```
/products + language=en  
/products + language=fr
```



can be treated as different variants.

---

## 6. What about cookies?

Here's where things get more nuanced.

You *can* design caching behavior around cookies, but you generally don't want to casually make:

```
Cache Key =  
URL + entire Cookie header
```



because imagine:

```
Cookie:  
session=abc123;  
theme=dark;  
cart=92837;  
...
```



Now every user's session could produce a completely different cache entry.

You'd destroy much of the benefit of shared caching.

You could theoretically get:

```
/products + userA cookie  
/products + userB cookie  
/products + userC cookie  
/products + userD cookie  
...
```



That's basically **personalized caching**.

---

## 7. This is why personalized pages are tricky

Imagine:

`</> TypeScript`

```
const session = cookies().get("session")

return <h1>Welcome, {sessionUser.name}</h1>
```



User A:

Welcome Gopi



User B:

Welcome Rahul



You absolutely cannot have:

```
/products
↓
Welcome Gopi
```



as one generic shared CDN response.

Otherwise:

```
User B
↓
CDN
↓
"Welcome Gopi"
```



😞 That's obviously terrible.

So systems generally avoid treating such personalized responses as ordinary shared public cache entries, or explicitly vary/cache them according to an appropriate strategy.

---

## 8. Now connect this to Next.js

This is why our earlier statement was:

```
cookies()
↓
```



```
request-dependent
↓
dynamic rendering
```

Under the **traditional Layer 1 model**, Next.js doesn't say:

"I'll just put this personalized page in the generic Full Route Cache."

Instead, the route is treated as request-dependent.

Conceptually:

```
Request
↓
Next.js
↓
cookies()
↓
render THIS request
↓
response
```



## 9. But here's a very important distinction 🚨

Don't think:

"If a request contains a cookie, CDN can never cache it."

That's too simplistic.

The correct idea is:

**The presence of a cookie doesn't automatically make every response uncacheable. What matters is whether the response is safe to reuse for another request.**

For example:

```
Cookie: theme=dark
```



but `/products` doesn't use theme.

The response could still potentially be shared.

Whereas:

Cookie: userId=123



and the response says:

Welcome Gopi



That's personalized.

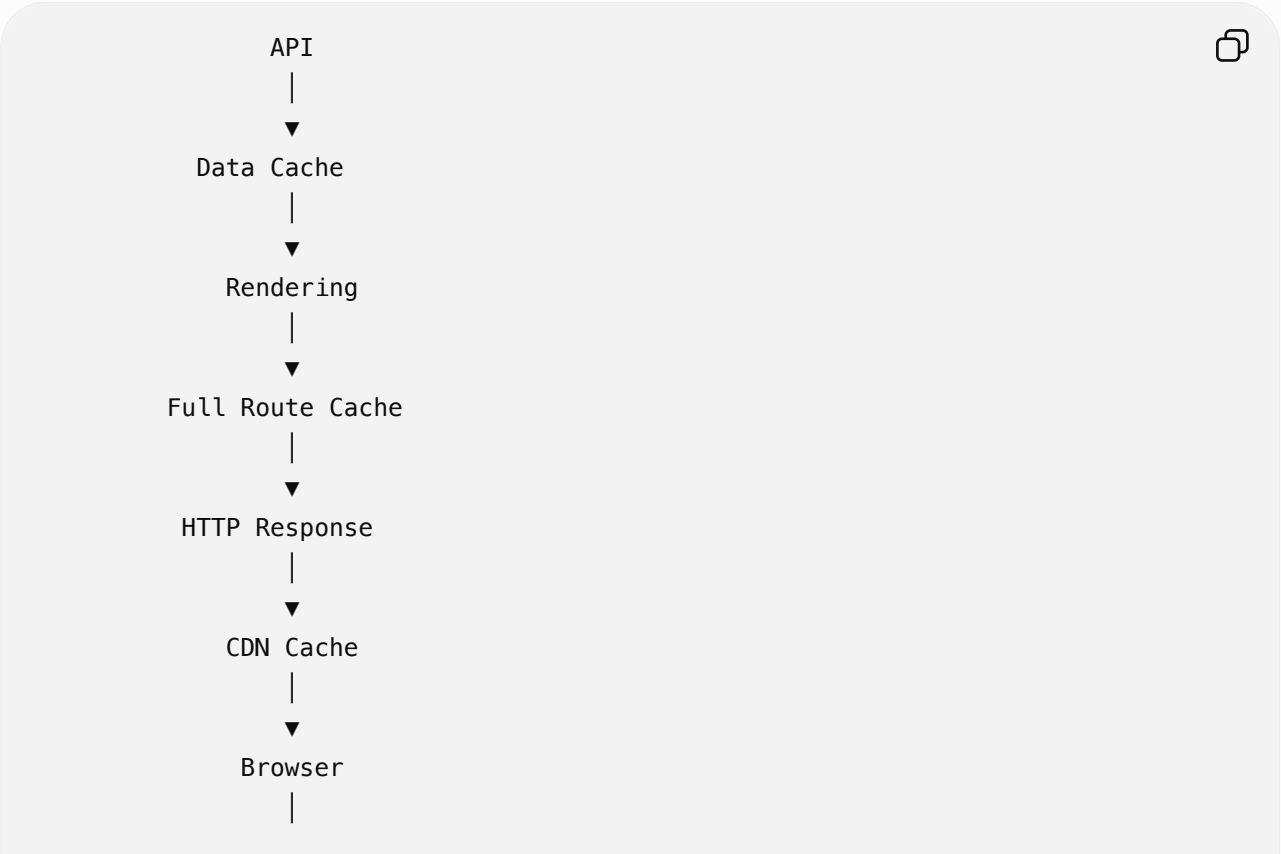
Different situation.

## 10. Now our three cache layers become clearer

Let's put them side-by-side.

| Layer            | What it caches        | Where                         |
|------------------|-----------------------|-------------------------------|
| Data Cache       | Fetches data          | Next.js/server infrastructure |
| Full Route Cache | Rendered route result | Next.js/server infrastructure |
| CDN/HTTP cache   | HTTP responses        | CDN/edge/intermediary         |
| Router Cache     | RSC payload           | Browser/client                |

So:



## Router Cache

And importantly, **these aren't duplicate copies of exactly the same thing for the same purpose.**

## 11. One killer example 🔥

Imagine your page:

```
</> TypeScript

export default async function Page() {
  const cookieStore = await cookies()
  const theme = cookieStore.get("theme")?.value

  const response = await fetch(
    "https://api.example.com/products",
    {
      next: { revalidate: 60 }
    }
  )

  const products = await response.json()

  return (
    <div className={theme}>
      ...
    </div>
  )
}
```

Now there are **two different kinds of information:**

### Cookie

theme=dark

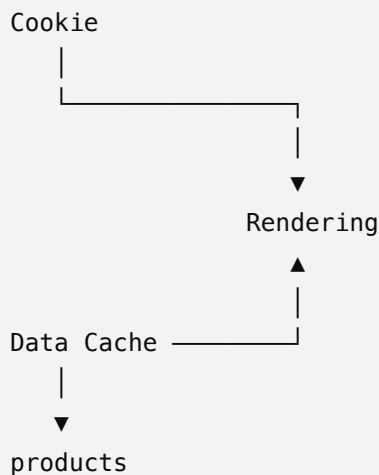
Request-specific.

### Products API data

iPhone  
MacBook  
AirPods

Potentially cacheable/reusable.

So conceptually:



This is a really important insight:

**One route can have request-specific information AND separately cacheable data dependencies.**

And this becomes **hugely important when we get to Layer 2 / Cache Components**, because modern Next.js gives you more precise control over which parts are cached and which parts remain request-dependent.

## Your mental model now

Don't think:

```

Cookie changed
  ↓
Update CDN cache
  
```

Instead:

```

Cookie changed
  ↓
Browser stores new cookie
  ↓
Future request
  ↓
Cookie automatically attached
  ↓
CDN receives request
  ↓
Can cached response safely represent this request?
  |
  | — YES → CDN can serve it
  
```

```
└─ NO → request may need origin/server
      ↓
    Next.js
      ↓
    cookies()
```

That's the whole game. 🎯

And now we're ready for the **Layer 1 debugging exercise**. I'll give you a route and code, and you tell me what happens. I'll deliberately mix **dynamic URL segments**, **cookies**, **searchParams**, **fetch**, **no-store**, **and** **revalidate** so you have to distinguish all of them.



ChatGPT can make mistakes. Check important info.